Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

A PROJECT TITLE ON

# Accident Hotspot Detection

## Using DBSCAN-Based Anomaly Detection

Project report submitted in partial fulfilment of the requirements for the course MACHINE LEARNING • BACHELOR OF TECHNOLOGY • A.Y. 2026-27

PRESENTED BY

**Bathula Neha** 2520090013
**Veeramallu Rohita Sai Lakshmi** 2520090235
**Koumudi** 2520090161

UNDER THE GUIDANCE OF

**Haritha**
Assistant Professor
Department of CSIT



# Abstract

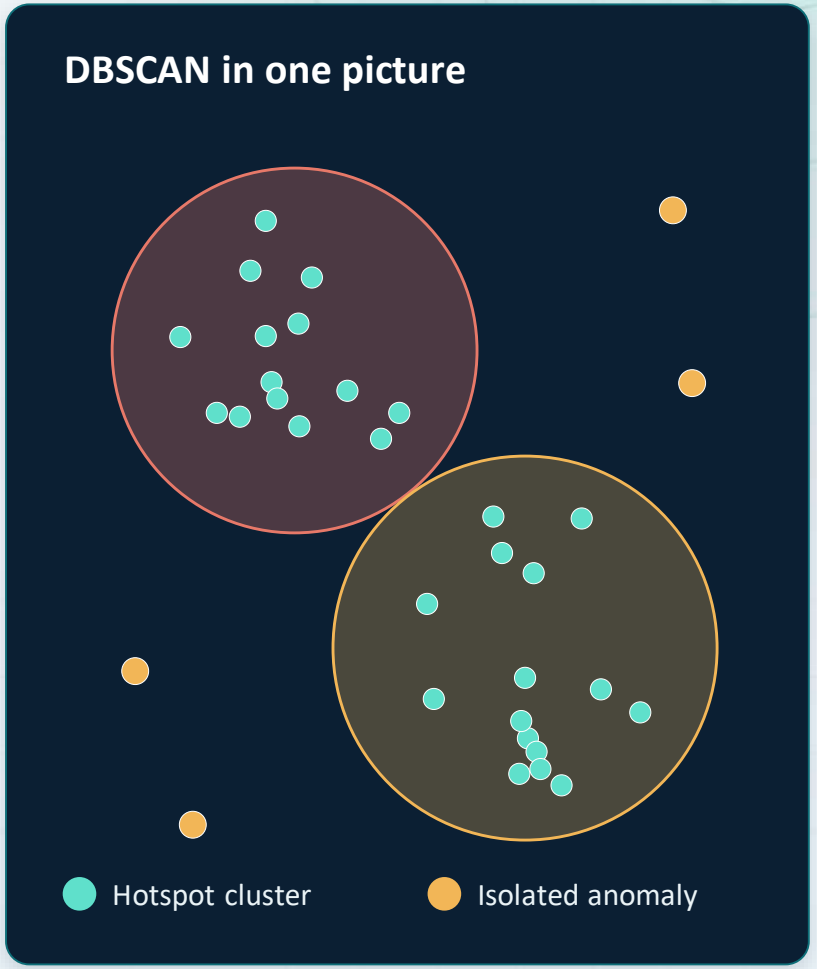
### The problem
Accidents recur at specific locations, but spotting these hotspots by hand is slow and error-prone.

### The method
DBSCAN, an unsupervised algorithm, groups accident-prone locations and flags isolated incidents as anomalies.

### The data
Public sources (US Accidents, UK Road Safety, Indian Road Accident); this work uses the Indian dataset, 2022–2025.

### The workflow
Collection, preprocessing, EDA, feature selection and DBSCAN, validated by Silhouette, Davies–Bouldin and expert review.

### The output
Locations labelled high, medium, low risk or anomaly, shown on interactive maps for safety planning.



# Introduction

## Background

Road traffic accidents remain a major public-safety challenge and often cluster at specific locations — poor road design, congestion, sharp turns, weak lighting and rule violations.

## Domain overview

Geospatial machine learning: geo-tagged accident records from government portals and open datasets let unsupervised algorithms find spatial patterns automatically.

## Why it is needed

Manual analysis of accident reports is slow, inconsistent and hard to scale across regions. Authorities need an automated, repeatable way to find hotspots.

**Goal:** a hotspot map that helps traffic authorities, urban planners and emergency services prioritise interventions and allocate resources effectively.

# Problem Statement

THE PROBLEM

**Accidents repeatedly strike the same places — yet finding those places by hand is slow, inconsistent and hard to scale.**



Recurring locations need targeted fixes — but first they must be found.

## Limitations & challenges

- Manual review of accident reports is slow and prone to oversight
- No consistent definition of a hotspot between analysts
- Does not scale to multi-year, multi-city geo-tagged records
- Isolated incidents get mixed up with true recurring hotspots

## Who is affected

Road users | Traffic authorities | Urban planners | Emergency services | Insurers

# Motivation

**Why we selected it**

Accident hotspots are a concrete, high-impact problem where unsupervised ML adds value without labelled data — and DBSCAN's noise handling suits messy geospatial records.

**Real-world need**

Every accident means lost lives, injuries and property damage. Authorities need to know where to act first, because repeat locations are where targeted fixes pay off most.

**Research & industry motivation**

Open geo-tagged datasets (US Accidents, UK Road Safety, Indian Road Accident) and smart-city programmes make data-driven road-safety analysis practical.

# Objectives

MAIN OBJECTIVE

**Build an ML-based system that automatically detects accident hotspots and isolates anomalous incidents using DBSCAN.**

1. Collect and preprocess geo-tagged accident records from a public dataset
2. Explore patterns by city, severity, time, cause and weather (EDA)
3. Apply city-wise DBSCAN with tuned eps and min_samples to find hotspots
4. Evaluate clusters with Silhouette, Davies–Bouldin and expert review
5. Classify risk levels and visualise results on an interactive dashboard

# Existing System

## Current approach

Analysts manually read accident reports, tally incidents per location and mark high-count spots on static maps.

## Existing technologies

- Spreadsheets and statistical summaries
- Static GIS or paper maps
- Centroid-based clustering such as K-Means

## Limitations

- Slow, labour-intensive and inconsistent between analysts
- Does not scale to large multi-year datasets
- K-Means needs the number of clusters in advance
- Centroid methods assume compact, round clusters
- Outliers are forced into clusters instead of flagged
- Static outputs — no interactive exploration

# Proposed System

ROADLENS

**An automated pipeline that runs city-wise DBSCAN on geo-tagged accident records and presents hotspots on an interactive dashboard.**

DBSCAN | Haversine | Flask

**Each limitation of the existing approach is addressed:**

| | |
|---|---|
| Manual, slow analysis | **Automated pipeline over geo-tagged records** |
| Cluster count fixed in advance | **DBSCAN finds hotspots without a preset k** |
| Outliers forced into clusters | **Isolated incidents flagged as noise (–1)** |
| Round-cluster assumption | **Density-based clusters of any shape** |
| Static reports | **Interactive map, charts and CSV export** |

# System Architecture




## INPUT

- CSV accident records (Kaggle)
- Latitude, longitude, city
- Severity, cause, weather, hour
- User parameters: min_samples, eps override



## PROCESSING

- Detect and validate columns
- Clean and range-check coordinates
- Degrees to radians
- Split records by city



## ALGORITHM

- City-wise DBSCAN
- Haversine distance
- City-specific eps (km)
- min_samples = 10, label −1 = noise



## OUTPUT

- Hotspot clusters and anomalies
- High / Medium / Low risk
- Silhouette and Davies–Bouldin
- Map, charts, CSV export

# Methodology

1

## Data collection

Indian Road Accident Dataset (Kaggle, 2022–2025)

2

## Preprocessing

Clean coordinates, parse fields, drop invalid rows

3

## EDA

City, severity, hour, cause, weather and traffic patterns

4

## Feature selection

Latitude and longitude drive clustering; other fields profile hotspots

5

## DBSCAN clustering

City-wise, Haversine distance, tuned eps, min_samples = 10

6

## Evaluation

Silhouette, Davies–Bouldin, noise share and expert review

7

## Risk labelling

High / Medium / Low by cluster size; isolated = anomaly

8

## Visualisation

Interactive map, charts, hotspot table and CSV export

# Technology Stack

| | |
|---|---|
| **Language** | Python 3.x |
| **Frameworks** | Flask, Scikit-learn |
| **Libraries** | Pandas, NumPy, Matplotlib, Seaborn, Folium, Plotly |
| **Front end** | HTML / CSS / JS, Chart.js, Leaflet.js |
| **Database** | CSV in / CSV out, No database server required |
| **Environment** | Jupyter / Colab / VS Code, Windows / Linux / macOS, Standard laptop |

# System Modules

**User Interface**
Flask dashboard: upload, parameters, city filter

**Data Collection**
Kaggle CSV upload or built-in synthetic demo

**Data Preprocessing**
Column detection, coordinate cleaning, radians

**Core ML Module**
City-wise DBSCAN with the Haversine metric

**Decision Module**
Relative High / Medium / Low risk labels

**Visualisation & Reporting**
Leaflet map, Chart.js charts, CSV export

**Database**
CSV files only — no database server

**Security & Validation**
CSV-only, 25 MB cap, parameter range checks, local processing

# Implementation

1. **Load CSV**
   Auto-detect latitude, longitude and city columns

2. **Clean**
   Numeric coercion, range check, drop missing rows

3. **Cluster**
   Run DBSCAN per city on radian coordinates

4. **Summarise**
   Merge labels, profile clusters, compute metrics

5. **Serve**
   Flask returns JSON for charts, map and table

scikit-learn • core of the model

```
coords = np.radians(city_df[['lat', 'lon']])
labels = DBSCAN(
    eps=eps_km / 6371.0,    # km to radians
    min_samples=10,
    metric='haversine',
    algorithm='ball_tree'
).fit_predict(coords)       # -1 = noise
```



Silhouette and Davies–Bouldin are computed on km-projected coordinates when at least two clusters exist.

# Dataset

**SOURCE**

**Kaggle — Indian Road Accident Dataset**

**PERIOD**

**2022 – 2025**

**CITIES**

**8 cities, clustered separately**

## Key attributes (used when present in the CSV)

**Location:** latitude, longitude, city, state

**Time:** date, hour, day of week, festival

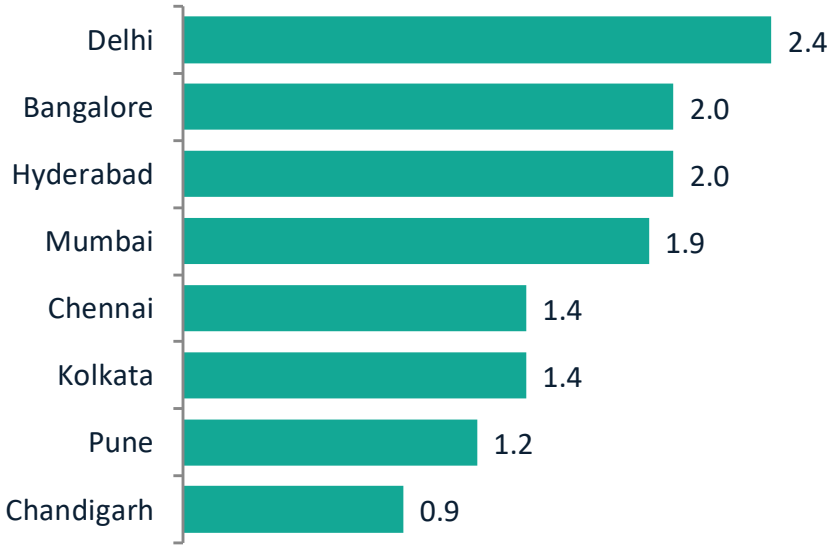
**Road & weather:** road type, weather, visibility, traffic density

**Accident:** severity, cause, vehicles, casualties

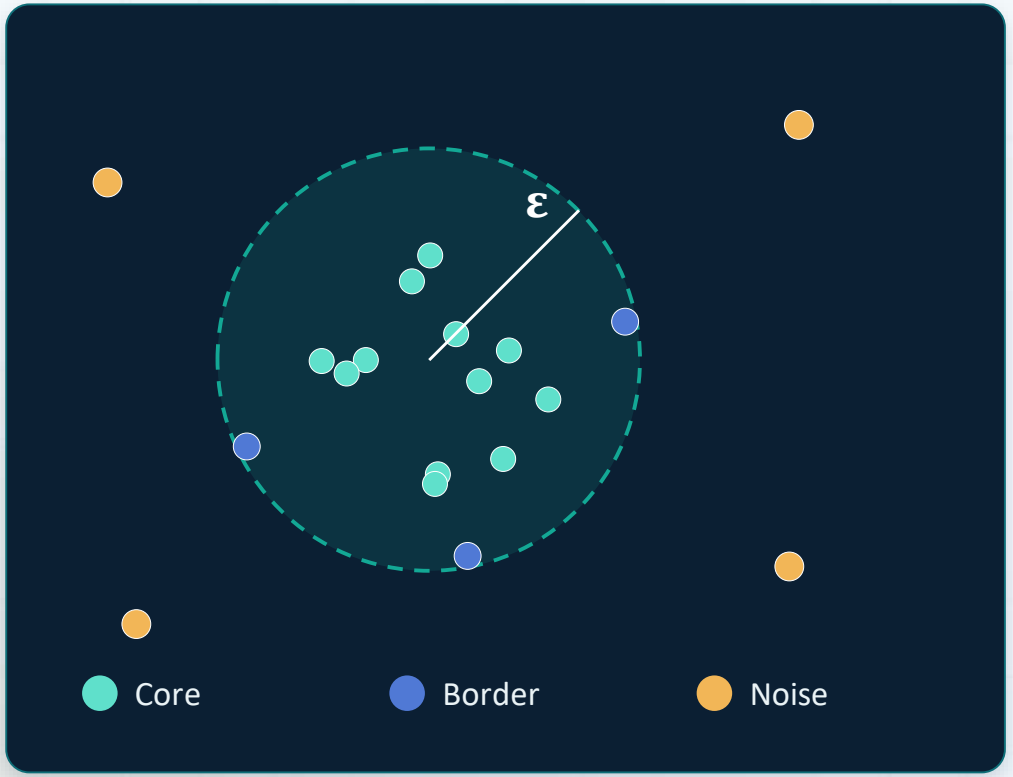
## Preprocessing

Coerce lat/lon to numbers • drop missing and out-of-range rows • trim city names • convert degrees to radians

## City-specific eps (km)



# Algorithm: DBSCAN




## How DBSCAN works

- Core: at least min_samples neighbours within ε
- Border: within ε of a core point, but not dense itself
- Noise: neither — flagged as an isolated anomaly (−1)

## Why it was selected

- No need to preset the number of clusters
- Finds irregular, non-spherical hotspot shapes
- Robust to noisy geospatial data; flags outliers

**ε (eps)** 0.9 – 2.4 km, per city

**min_samples** 10

**Metric** Haversine (great-circle)

# Results & Evaluation

| Metric | What it tells | How to read it | Final run |
|---|---|---|---|
| **Clusters detected** | Number of hotspot clusters | Count | *add value* |
| **Noise points** | Isolated records (label −1) | Share of records, % | *add value* |
| **Silhouette Score** | Cohesion vs. separation | Closer to 1 is better | *add value* |
| **Davies–Bouldin Index** | Overlap between clusters | Lower is better | *add value* |
| **Expert review** | Match with known black spots | Qualitative | *add value* |

Results describe historical spatial concentrations — not predictions of future crashes. Risk levels are relative to cluster sizes.

## Why not accuracy, precision, recall or F1?

DBSCAN is unsupervised: accident records carry no ground-truth cluster labels, so supervised scores do not apply.

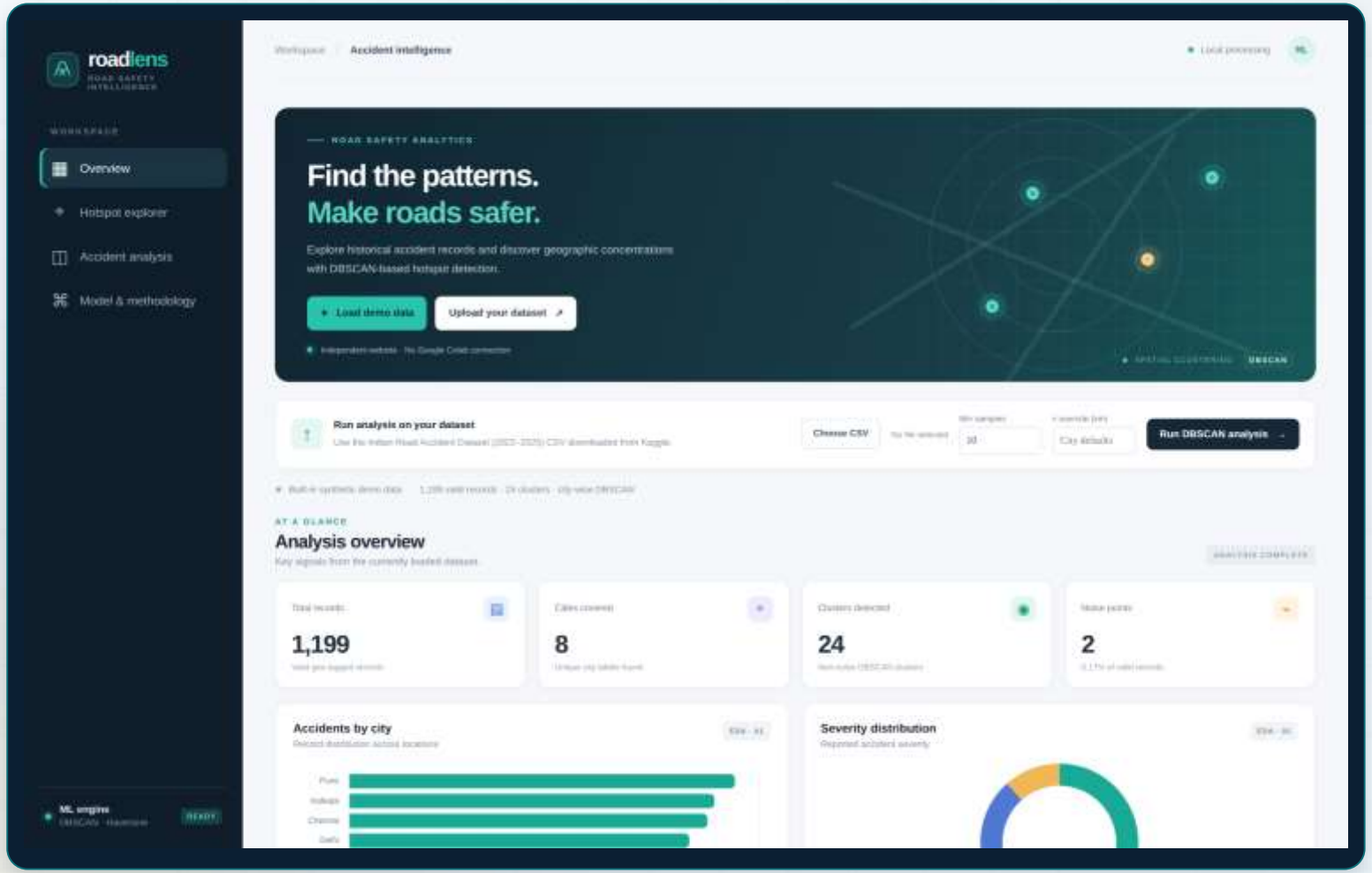
Cluster quality is judged with internal indices (Silhouette, Davies–Bouldin), noise share, map inspection and expert review.

# Comparison with Existing Methods

| Criterion | DBSCAN (proposed) | K-Means | Manual analysis |
|---|---|---|---|
| **Number of clusters** | **Not required** | Set in advance | Not applicable |
| **Cluster shape** | **Arbitrary shapes** | Roughly round only | Subjective |
| **Isolated incidents** | **Flagged as noise** | Forced into a cluster | Judgement call |
| **Scalability** | **High, automated** | High, automated | Low, labour-intensive |
| **Consistency** | **Repeatable** | Repeatable, init-sensitive | Varies by analyst |
| **Hotspot meaning** | **Dense areas on a map** | Centroid-based regions | Count-based ranking |

Qualitative comparison based on the properties of each approach.

# Demo: Home & Input Screens




**1 Home page**
Hero, navigation and a built-in demo dataset

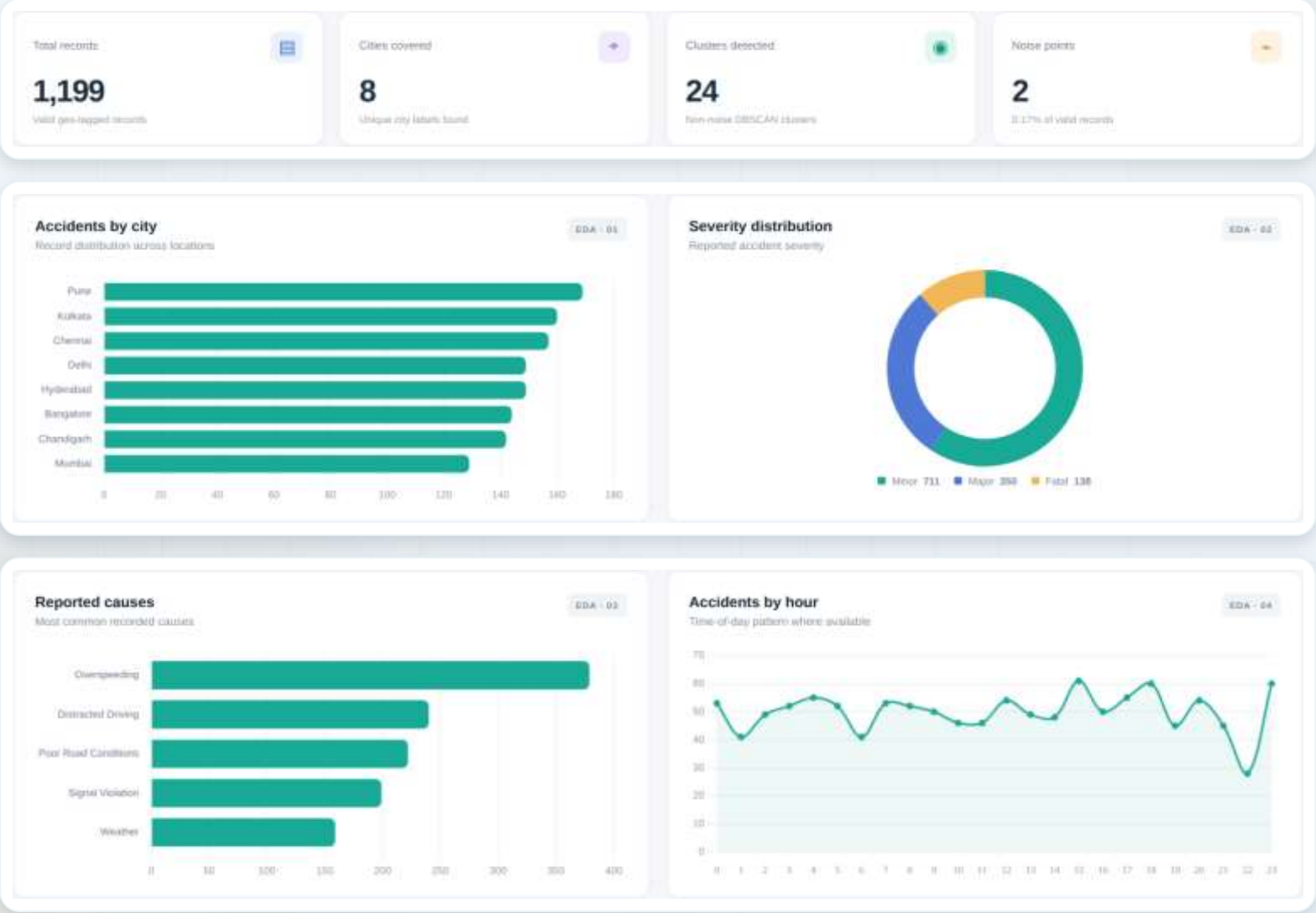
**2 Input screen**
Upload CSV, set min_samples and optional eps

**3 Processing**
'Run DBSCAN analysis' clusters each city

**4 Output screen**
Metrics, charts, map and hotspot table

**Synthetic demo data — illustration only**

# Demo: Analysis Dashboard



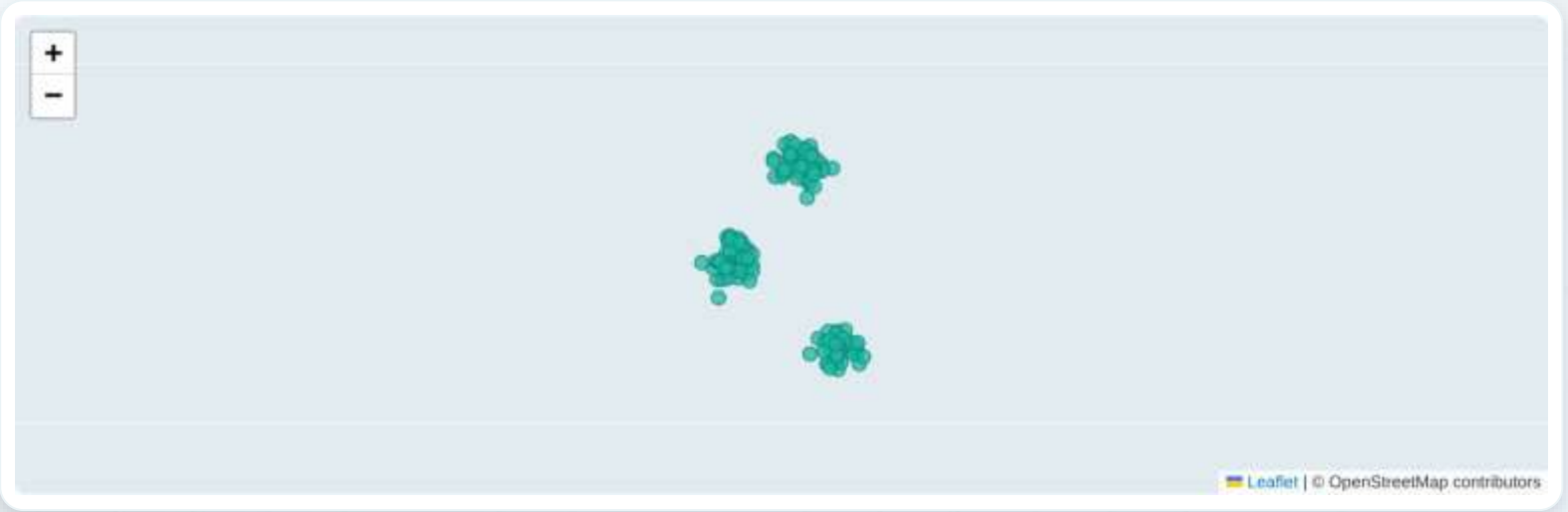

**Overview metrics**
Records, cities, clusters and noise at a glance

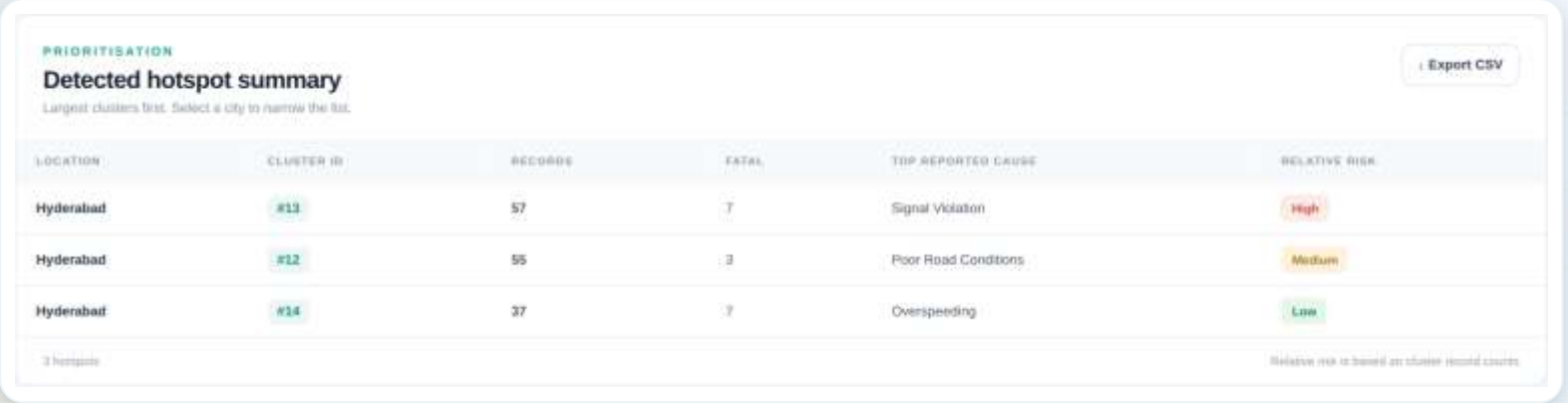
**EDA charts**
City, severity, cause, hour, density and weather

**Live recompute**
Change parameters and re-run the analysis

**Synthetic demo data — illustration only**

# Demo: Hotspot Map & Summary




**Hotspot explorer**
Clusters vs noise points, filter by city

**Hotspot summary**
Cluster id, records, fatal, top cause, relative risk

**Export**
Download detected hotspots as CSV

**Synthetic demo data — illustration only**

# Advantages

### No preset cluster count

DBSCAN discovers the number of hotspots from the data itself.

### Any hotspot shape

Follows roads and corridors instead of forcing round clusters.

### Anomaly-aware

Isolated incidents are flagged as noise, not forced into hotspots.

### Automated and scalable

Replaces slow manual review; city-wise runs handle large data.

### Actionable outputs

Risk levels, maps and a CSV make prioritisation simple.

### Open and low cost

Free Python tools; runs on a standard laptop.

# Applications

**Smart-city traffic management**
Prioritise junctions and corridors

**Road-safety analysis**
Find recurring accident locations

**Accident-prevention planning**
Target engineering and enforcement

**Emergency-response optimisation**
Position resources near hotspots

**Insurance risk assessment**
Inform location-based risk models

TARGET USERS

- Traffic authorities and road agencies
- Urban and transport planners
- Emergency services
- Insurance companies
- Researchers and students

# Limitations

## Current limitations

- Shows historical concentrations, not predictions of future crashes
- Risk levels are relative to cluster sizes — not calibrated probabilities
- Results are sensitive to the eps and min_samples choices
- No live data: hotspots update only when a new CSV is run

## Dataset & technical constraints

- Depends on reporting coverage and quality of geo-tags
- Records without valid latitude/longitude are dropped
- Silhouette / DBI need 2+ clusters and ≤ 5,000 clustered points
- Map view capped at 12,000 points; map tiles need internet

# Future Scope



1

## Real-time reporting

Ingest accident reports as they happen.

2

## Dynamic hotspots

Combine live GPS, weather and traffic data.

3

## Web & mobile app

Cloud-hosted app for authorities and citizens.

4

## Deep learning

Predict accident severity for each hotspot.

5

## Safer routing

Recommend alternative routes that avoid hotspots.

# Conclusion

### What was achieved

An end-to-end pipeline — cleaning, EDA, city-wise DBSCAN and evaluation — with an interactive RoadLens dashboard.

### Key outcome

Recurring accident hotspots and isolated anomalies are separated automatically, without presetting the number of clusters.

### Overall contribution

A reusable, low-cost and explainable approach that helps authorities prioritise road-safety interventions.

## Find the patterns. Make roads safer.

# Thank You

## Questions & Discussion

Bathula Neha • 2520090013

Veeramallu Rohita Sai Lakshmi • 2520090235

Koumudi • 2520090161

Guide: Haritha, Assistant Professor, Dept. of CSIT

REFERENCES


- Ester, Kriegel, Sander, Xu (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. KDD-96.
- Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12.
- Indian Road Accident Dataset 2022–2025, Kaggle (sehaj1104).